Project Plan Document (Jackfruit Phase-2)

Face Recognition-Based Smart Attendance Management System

Project ID: 20

Version 1.0

UE24CS341A - Software Engineering, Jackfruit Mini-Project

Team Details

Field	Details
Team Name	<Team Name>
Team Member 1	<Name> | SRN: <SRN>
Team Member 2	<Name> | SRN: <SRN>
Team Member 3	<Name> | SRN: <SRN>
Team Member 4	<Name> | SRN: <SRN>
Prepared by	<Author Name(s)>
Date Created	<DD Month 2026>

Revision History

Name	Date	Reason For Changes	Version
<Name>	<Date>	Initial draft	1.0

Table of Contents

(Right-click the table above and choose “Update Field” in Word to populate page numbers.)

1. Introduction

This Project Plan covers how the team will execute the implementation of the Face Recognition-Based Smart Attendance Management System (Project ID 20), following on from the completed SRS, Architecture, SW Design, and Validation Spec documents. It defines the lifecycle model, tooling, build-vs-reuse decisions, work breakdown, effort estimates, and schedule, as required for Jackfruit Phase-2.

2. Lifecycle Model and Justification

The team will follow Agile (Scrum-style, short sprints), as mandated by the course guidelines. Concretely: work is broken into the WBS tasks in Section 5 below, tracked as GitHub Issues on a GitHub Projects board (To Do / In Progress / Done), with a short weekly check-in among team members standing in for a sprint review (matching the course's own weekly evaluation cadence).

This model was chosen over a Waterfall approach because:

- The SRS (Section 2.6, Assumptions and Dependencies) already flags real uncertainty – e.g. how well LBPH recognition will perform in practice is only known once it is tried, so requirements for the confidence threshold and override behaviour benefit from being refined iteratively rather than frozen upfront.
- Each of the 4 team members owns one functional feature end-to-end (Section 5), which maps naturally onto parallel, independent sprints/backlogs rather than sequential phases.
- The course's own weekly, contribution-based evaluation (graded via git commits) rewards steady incremental delivery, which Agile supports directly and Waterfall does not.

3. Tools Used Across the Lifecycle

Lifecycle stage	Tool	Justification
Planning / backlog & agile board	GitHub Issues + GitHub Projects (Kanban board)	Already required for the course (‘Github actively used for user stories, backlog’); no separate Jira account needed for a 4-person team.
Design / diagramming	draw.io (diagrams.net) or the equivalent used to produce this document's diagrams	Free, browser-based, exports PNG/SVG directly into Word/Markdown docs.
Distributed version control	Git	Mandated by guidelines.
Version control hosting	GitHub (private org repo)	Mandated by guidelines; also hosts Issues/Projects/Actions in one place.
Development environment	VS Code + Python virtual environments (venv)	Free, cross-platform, already in use by the team (see earlier setup).
Bug tracking	GitHub Issues (labelled ‘bug’)	Same tool as planning – avoids maintaining a second system.
Testing tools	Python's unittest / pytest for unit tests; Flask's built-in test client for integration tests; manual test logs for camera-dependent flows	Matches the Python/Flask stack; camera-dependent flows cannot be fully automated (see Validation Spec Section 2).
CI pipeline	GitHub Actions (run pytest on every push/PR)	Satisfies the course's CI/CD requirement with zero extra hosting cost.

Lifecycle stage	Tool	Justification
Code review	GitHub Pull Request reviews (at least one teammate approval before merging to main)	Mandated by guidelines (‘code reviews... mandatory’); enforced via a simple team norm given the small team size.
Static code analysis	flake8 / pylint (run locally or as a CI Action step)	Lightweight, free, catches basic issues (unused imports, style) before review.
LLM assistance	Claude (or similar), used for scaffolding code, debugging, and documentation	Explicitly permitted by the guidelines; every team member must still be able to explain any LLM-assisted code in the final presentation.

4. Deliverables: Reuse vs. Build

Classifying each major component clarifies where the team's own engineering effort goes, versus what is adopted as-is from existing open-source work – relevant both for effort estimation (Section 6) and for being able to explain, in the final presentation, exactly what the team built.

Component	Reuse / Build	Justification
Flask web framework	Reuse	Standard, well-documented web framework; reimplementing it would add no value.
OpenCV (Haar cascade + LBPH recognizer)	Reuse	A pretrained, general-purpose face detector and a standard recognizer algorithm; training a face-recognition model from scratch is out of scope for a mini-project.
Bootstrap CSS framework	Reuse	Off-the-shelf UI styling; not a differentiating part of this project.
SQLite engine	Reuse	Standard embedded database; only the schema (Section 5 of SW Design Doc) is the team's own design.
Database schema (Student, Attendance tables)	Build	Designed specifically for this system's requirements (SRS Appendix B).
Flask routes / controllers	Build	Application-specific request handling, written by the team.
Face enrollment & recognition service logic	Build (on top of reused libraries)	The specific capture flow, sample-count rules, confidence threshold handling, and error handling (SW Design Doc Section 9) are the team's own logic layered on OpenCV's primitives.
Analytics computation & CSV export	Build	Attendance-percentage logic and report formatting are specific to this project's requirements.
Auth / role-based access control	Build	Application-specific login and permission checks across Admin/Faculty/Student roles.
HTML templates / UI screens	Build	Page layout and content are project-specific, even though they use the reused Bootstrap components.

5. Work Breakdown Structure (WBS)

Tasks are grouped by the same features defined in the SRS (Section 5) and assigned so each of the 4 team members owns one full functional feature end-to-end, as required by the course guidelines.

5.1 Project management & documentation (shared)

- WBS-0.1 Software Requirements Specification (complete)
- WBS-0.2 High-Level Architecture Document (complete)
- WBS-0.3 SW Design Document (complete)
- WBS-0.4 Validation Spec update (complete)
- WBS-0.5 This Project Plan Document
- WBS-0.6 Phase-3 Test Plan Document (upcoming)
- WBS-0.7 Final report compilation and 15-minute demo rehearsal

5.2 Member A – Face Enrollment module

- WBS-1.1 Implement capture_face_samples() (webcam capture + Haar cascade detection)
- WBS-1.2 Implement train_model() (LBPH training/retraining)
- WBS-1.3 Enroll route + form + roster display template
- WBS-1.4 Unit tests for enrollment edge cases (0 faces, 2+ faces, duplicate roll number)

5.3 Member B – Recognition & Attendance module

- WBS-2.1 Implement get_face_detector() / recognize_and_mark()
- WBS-2.2 Attendance marking + manual override UI and route
- WBS-2.3 Confidence-threshold tuning and 'Unknown' handling
- WBS-2.4 Unit/integration tests for attendance marking and override

5.4 Member C – Analytics & Reporting module

- WBS-3.1 Attendance-percentage computation (per student, per section)
- WBS-3.2 Dashboard template with below-75% flagging
- WBS-3.3 CSV export route
- WBS-3.4 Unit tests for percentage calculation and CSV formatting

5.5 Member D – Auth/Roles & overall UI

- WBS-4.1 Login, password hashing, session management
- WBS-4.2 Role-based route protection (Admin/Faculty/Student)
- WBS-4.3 Shared base template, navigation, styling consistency across all pages
- WBS-4.4 Integration tests for access control (REQ-AU, REQ-OV-3, REQ-AN-4)

5.6 Shared – Integration, testing, and delivery

- WBS-5.1 Merge all four modules on a shared `main` branch via reviewed pull requests
- WBS-5.2 End-to-end manual test of the full enroll -> attendance -> dashboard -> export flow
- WBS-5.3 System validation against the Validation Spec (Section 3 mapping)

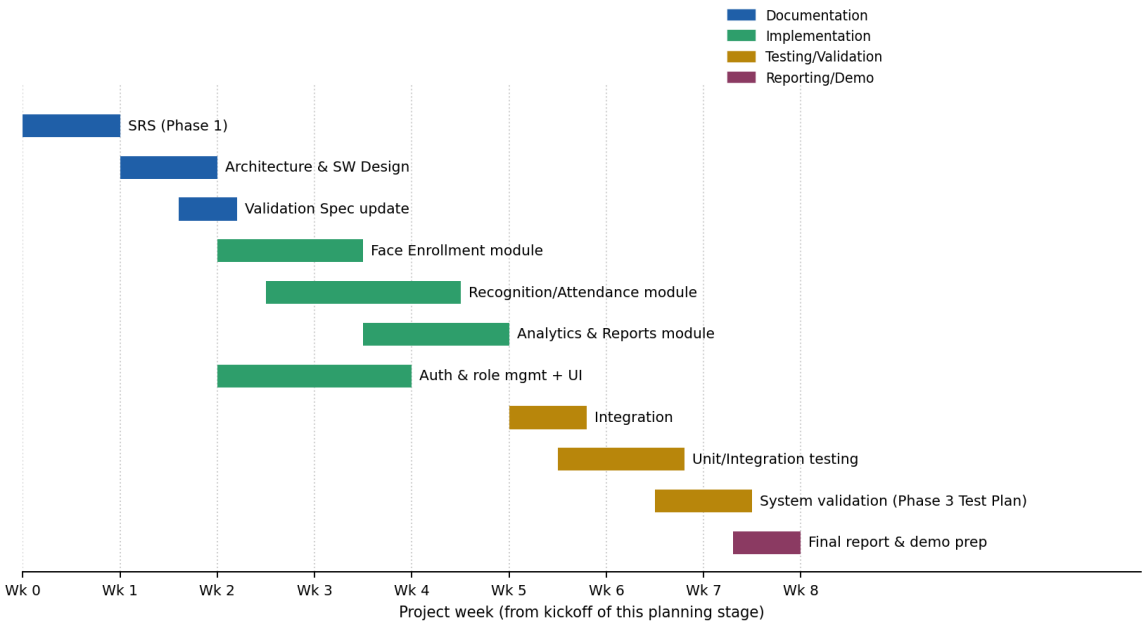
- WBS-5.4 Final report assembly and demo rehearsal

6. Effort Estimation (Person-Months)

Estimates assume a part-time student effort (roughly 8-10 hours/week per person alongside other coursework) over the remaining project weeks; 1 person-month here is approximated as ~20 working days of part-time effort, consistent with the small scope of a mini-project.

WBS item	Owner	Estimated effort (person-months)
0. Documentation (SRS, Architecture, SW Design, Validation Spec, Plan)	Shared (already largely complete)	0.6
1. Face Enrollment module	Member A	0.4
2. Recognition & Attendance module	Member B	0.5
3. Analytics & Reporting module	Member C	0.35
4. Auth/Roles & overall UI	Member D	0.35
5. Integration, testing & delivery	Shared	0.4
Total		2.6

7. Schedule (Gantt Chart)



Weeks are relative to the start of this planning stage; the team should map 'Wk 0' onto their actual calendar date and adjust against any faculty-assigned deadlines for Phase-3 and the Final Demo. Documentation tasks (navy) are already complete per this and the preceding Phase-1 submission; implementation of the four modules (green) runs partly in parallel since each is owned by a different team member; integration and testing (gold) deliberately starts only once at least the Enrollment and Recognition modules are functional, since later modules depend on having attendance data to analyze and report on.